

NAME: SHAIK ARSHAD BASHA

REG NO: 192312336

1. A simple perceptron model is used to classify whether a student passes or fails based on two input features, namely study hours and attendance. The weights associated with these inputs are 0.6 and 0.4 respectively, and the bias value is -0.5 . For a given student with study hours equal to 2 and attendance equal to 1, compute the net input to the perceptron and apply the step activation function to determine the final output. Based on the result, conclude whether the student passes or fails.

A) 1. Conceptual Understanding

A perceptron is a basic artificial neural network used for binary classification. It takes input features, multiplies them by their corresponding weights, adds a bias, and then applies an activation function.

In this problem, the perceptron determines whether a student passes or fails based on:

- Study Hours = 2
- Attendance = 1
- Weight for Study Hours = 0.6
- Weight for Attendance = 0.4
- Bias = -0.5

2. Accuracy of Answer

Substituting the given values:

Substituting the given values:

$$Net = (2 \times 0.6) + (1 \times 0.4) + (-0.5)$$

$$Net = 1.2 + 0.4 - 0.5$$

$$\boxed{Net = 1.1}$$

Therefore, the **net input to the perceptron is 1.1**.

3. Application of Knowledge

The step activation function is defined as:

The **step activation function** is defined as:

$$f(Net) = \begin{cases} 1, & Net \geq 0 \\ 0, & Net < 0 \end{cases}$$

Since the calculated net input is:

$$Net = 1.1$$

and

$$1.1 \geq 0$$

the perceptron produces:

$$\boxed{Output = 1}$$

Here, 1 represents Pass and 0 represents Fail.

Conclusion

The perceptron receives study hours = 2 and attendance = 1. After applying the weights and bias, the net input is 1.1. Since the net input is positive, the step activation function gives an output of 1.

$$\boxed{\text{Final Output} = 1}$$

Therefore, the student passes according to the perceptron model.

2. In a neural network, a single neuron is trained using the backpropagation learning rule. The initial weight is 0.5 and the learning rate is 0.1. For an input value of 2, the actual output produced by the neuron is 0.6, whereas the target output is 1. Calculate the error in the output and determine the weight update using the delta rule. Hence, compute the new updated weight after one iteration of training.

A) Backpropagation Learning Rule

1. Conceptual

The backpropagation learning rule adjusts the weight of a neuron based on the difference between the target output and the actual output.

The delta rule is:

$$\Delta w = \eta (t - y) x$$

where:

- Δw = weight change
- η = learning rate
- t = target output
- y = actual output
- x = input

2. Given Values

- Initial weight, ($w = 0.5$)
- Learning rate, ($\eta = 0.1$)

- Input, ($x = 2$)
- Actual output, ($y = 0.6$)
- Target output, ($t = 1$)

3. Calculate the Output Error

The output error is:

$$Error = Target - Actual$$

$$Error = 1 - 0.6$$

$$\boxed{Error = 0.4}$$

Therefore, the output error is 0.4.

4. Calculate Weight Update Using Delta Rule

Using the delta rule:

$$\Delta w = \eta(t - y)x$$

Substitute the values:

$$\Delta w = 0.1(1 - 0.6)(2)$$

$$\Delta w = 0.1(0.4)(2)$$

$$\Delta w = 0.08$$

Therefore:

$$\boxed{\Delta w = 0.08}$$

The positive value indicates that the weight should be increased.

5. Calculate New Updated Weight

The updated weight is

$$w_{new} = w_{old} + \Delta w$$

$$w_{new} = 0.5 + 0.08$$

$$w_{new} = 0.58$$

6. Conclusion

The error between the target and actual output is 0.4. Using the delta rule, the weight update is +0.08. Therefore, after one training iteration, the new updated weight is 0.58.

3. A deep learning model is developed for disease prediction and its performance is evaluated using a confusion matrix. The model yields 180 true positives, 150 true negatives, 20 false positives, and 50 false negatives. Using these values, compute the accuracy, precision, sensitivity (recall), and specificity of the model. Based on the calculated results, discuss whether the model is reliable for medical diagnosis, considering the importance of minimizing diagnostic errors.

A) Confusion Matrix Analysis for Disease Prediction

1. Conceptual Understanding

A confusion matrix is used to evaluate the performance of a classification model, especially in medical diagnosis.

The four outcomes are:

- True Positive (TP) = 180: Diseased patients correctly identified as diseased.
- True Negative (TN) = 150: Healthy patients correctly identified as healthy.

- False Positive (FP) = 20: Healthy patients incorrectly identified as diseased.
- False Negative (FN) = 50: Diseased patients incorrectly identified as healthy.

Given:

Total cases:

$$Total = 180 + 150 + 20 + 50 = 400$$

2. Accuracy

Accuracy measures the overall percentage of correct predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100$$

Substituting:

$$Accuracy = \frac{180 + 150}{400} \times 100$$

$$= \frac{330}{400} \times 100$$

$$\boxed{Accuracy = 82.5\%}$$

Therefore, the model correctly classifies 82.5% of all cases.

3. Precision

Precision measures how many patients predicted as diseased are actually diseased.

$$Precision = \frac{180}{180 + 20} \times 100$$

$$= \frac{180}{200} \times 100$$

$$\boxed{Precision = 90\%}$$

This means 90% of the patients predicted as positive actually have the disease.

4. Sensitivity (Recall)

Sensitivity measures how effectively the model identifies patients who actually have the disease.

$$\begin{aligned} \text{Sensitivity} &= \frac{180}{180 + 50} \times 100 \\ &= \frac{180}{230} \times 100 \\ \boxed{\text{Sensitivity} \approx 78.26\%} \end{aligned}$$

This means the model correctly detects approximately 78.26% of actual disease cases.

The remaining 21.74% of actual disease cases are missed as false negatives.

5. Specificity

Specificity measures how effectively the model identifies healthy patients.

Specificity measures how effectively the model identifies healthy patients.

$$\begin{aligned} \text{Specificity} &= \frac{TN}{TN + FP} \times 100 \\ \text{Specificity} &= \frac{150}{150 + 20} \times 100 \\ &= \frac{150}{170} \times 100 \\ \boxed{\text{Specificity} \approx 88.24\%} \end{aligned}$$

Therefore, the model correctly identifies approximately 88.24% of healthy patients.

Conclusion

The model shows reasonably good overall performance, with an accuracy of 82.5%. However, it cannot be considered fully reliable for medical diagnosis without further improvement, mainly because its sensitivity is relatively low.

In medical applications, false negatives can be more dangerous than false positives, because a missed disease may delay treatment. Therefore, the model should be improved to increase sensitivity/recall, while maintaining high specificity and precision.

Model performance is good, but sensitivity should be improved before clinical use.

4.A Genetic Algorithm is applied to maximize the function $f(x) = x^2$ using an initial population consisting of the values 2, 4, 6, and 8. Determine the fitness value for each individual in the population and select the best candidates based on their fitness scores. Perform a crossover operation by exchanging the least significant bits of the selected individuals to generate new offspring. Additionally, explain the role of mutation in improving the diversity and effectiveness of the solution in Genetic Algorithms.

A) Genetic Algorithm – Maximization of $f(x)=x^2$

1. Conceptual Understanding

A Genetic Algorithm (GA) is an optimization technique inspired by natural selection. It works through:

1. Population initialization
2. Fitness evaluation
3. Selection
4. Crossover

5. Mutation

Here, the objective is to maximize:

$$f(x) = x^2$$

Initial population:

2, 4, 6, 8

2. Fitness Calculation

The fitness value of each individual is calculated using:

$$Fitness = f(x) = x^2$$

Individual x	Fitness x^2
2	$2^2 = 4$
4	$4^2 = 16$
6	$6^2 = 36$
8	$8^2 = 64$

Therefore, the fitness values are:

4, 16, 36, 64

3. Selection of Best Candidates

Since the objective is to maximize x^2 , individuals with higher fitness are preferred.

Fitness ranking:

$$8 > 6 > 4 > 2$$

Therefore:

- Best candidate: $8 \rightarrow \text{Fitness} = 64$
- Second-best candidate: $6 \rightarrow \text{Fitness} = 36$

Thus, we select 6 and 8 as the parents for crossover.

4. Crossover Operation

Represent the selected individuals in binary:

$$6 = (0110)_2$$

$$8 = (1000)_2$$

The question specifies exchanging the least significant bit (LSB).

- $6 \rightarrow 0110$
- $8 \rightarrow 1000$

Exchange the last bit:

Parent1: 0110

Parent 2: 1000

Since both parents have the same LSB (0), exchanging the LSB produces no change.

Therefore, the offspring are:

$$\boxed{0110 = 6}$$

$$\boxed{1000 = 8}$$

So, in this particular crossover, the offspring remain 6 and 8.

Conclusion

The Genetic Algorithm evaluates each individual using the fitness function $f(x) = x^2$. Among the initial population, 8 has the highest fitness of 64, followed by 6 with a fitness of 36, so they are selected for crossover. Exchanging their LSBs produces no change because both have an LSB of 0. Mutation is essential for maintaining diversity and exploring new solutions, helping the Genetic Algorithm avoid local optima and improve its effectiveness.

Best initial solution = 8, Maximum fitness = 64

5. In a neural network, a single neuron uses the delta learning rule. The initial weight is 0.4, learning rate is 0.2, and input is 3. The actual output is 0.5, while the target output is 0.9. Calculate the error, determine the weight update, and compute the new weight after one iteration.

A) Delta Learning Rule: The Delta Learning Rule, also called the Widrow-Hoff learning rule, is used to update the weights of a neuron based on the difference between the target output and the actual output.

The main purpose of the learning rule is to reduce the error between the desired output and the output produced by the neuron.

1. The weight update equation is:

$$\Delta w = \eta(t - y)x$$

where:

- Δw = change in weight
- η = learning rate
- t = target output

- y = actual output
- x = input

2. Given Data

The problem gives:

Initial weight $w = 0.4$

Learning rate $\eta = 0.2$

Input $x = 3$

Actual output $y = 0.5$

Target output $t = 0.9$

3. Calculate the Output Error

The output error is the difference between the target output and actual output.

$$Error = t - y$$

Substituting the given values:

$$Error = 0.9 - 0.5$$

$$\boxed{Error = 0.4}$$

Therefore, the neuron has an output error of 0.4.

The positive error indicates that the actual output is lower than the desired target output.

4. Calculate the Weight Update

The Delta Learning Rule is:

$$\Delta w = \eta(t - y)x$$

Substitute the values:

$$\Delta w = 0.2(0.9 - 0.5)(3)$$

First calculate the error:

$$0.9 - 0.5 = 0.4$$

Therefore:

$$\Delta w = 0.2(0.4)(3)$$

$$\Delta w = 0.08 \times 3$$

$$\boxed{\Delta w = 0.24}$$

Thus, the weight must be increased by 0.24.

5. Calculate the New Weight

The new weight is calculated as:

$$w_{new} = w_{old} + \Delta w$$

Substituting:

$$w_{new} = 0.4 + 0.24$$

$$\boxed{w_{new} = 0.64}$$

Therefore, after one iteration of training, the new weight is 0.64.

6. Interpretation of the Result – 2 Marks

Initially, the neuron has a weight of 0.4 and produces an actual output of 0.5, whereas the desired target is 0.9.

Since:

$$0.5 < 0.9$$

the neuron needs to increase its output. The Delta Rule therefore produces a positive weight update of 0.24.

The weight changes from:

$$0.4 \rightarrow 0.64$$

This adjustment helps the neuron move its output closer to the target value.

Conclusion

Using the Delta Learning Rule, the error between the target and actual output is 0.4. With a learning rate of 0.2 and input of 3, the weight update is 0.24. Therefore, the new weight after one training iteration is:

$$w_{new} = 0.64$$

This demonstrates how the Delta Rule adjusts the neuron's weight to reduce the output error and improve learning performance.